CS126 Coursework Report

Introduction:

The Warwick+ application had 3 data stores (Movies, Credits, and Ratings), where each one stored and searched a different category of film data. The main dilemma was choosing data structures that supported fast lookup by unique integer key (TMDB film ID or user ID), whilst still being relatively memory-efficient across the dataset (consisting of approx. 1000 films and 16690 ratings). This report's aim is to explain the reasoning behind the data structures chosen and key implementation decisions made.

Key Data Structure: Hashtable with Separate Chaining:

All of the stores use hash tables as the main data structure, as every major operation ultimately simplifies to requiring us to find some kind of record with an integer key provided. There is O(1) average-case time for insertion, deletion, and lookup for hashtables compared to O(n) scan, which is needed for plain arrays or linked lists, so hash tables outperform for this kind of implementation.

The hash table uses the standard separate chaining approach. The backing structure is a fixed-length (/capacity given) array of node references; each slot stores the head of a singly-linked list. If two unique keys produce the same hash code (this is a collision), the new node replaces the existing head of the linked list, and the existing head is linked to the end of the new node (instead of an open addressing approach of searching for empty slots elsewhere). There's the benefit of separate chaining being able to handle high load factors more gracefully than open addressing, and allows chains to grow longer rather than clustering. The removal of a node is also straightforward, as a linked list is capable of bridging the gap caused by removal. The reason for inserting a node at the head of the chain was that it prevented having to traverse (so it was more time efficient).

The hash table capacity was 1999 for all 3 stores. This was a deliberate prime number choice, as a prime modulus decreases the probability of systematic collisions: it has no common factors with typical key distributions (allowing for an even spread of entries across the buckets).

As there are 1000 films, the load factor is approximately. 0.5, so on average, each bucket contains at most one or two entries. This keeps expected chain traversal relatively close to O(1) in practice. The hash function itself is minimal: the absolute value of the key modulo the capacity. Taking the absolute value guards against negative TMDB IDs, preventing a negative array index.

Movies Store:

The Movies store contains one record per film, accessed with TMDB film ID. Every record contains all the core metadata fields (i.e., title, overview, release date, budget, revenue, runtime, etc.), and optional fields were set lazily after the node's first insertion (IMDb ID, popularity, vote data, collection membership). As all fields are stored inside the node, it allows for data to be immediately accessible (without more than one lookup), as we just need to find the correct node using a hash lookup. O(1) on average for inserting a film, as we just hash the ID and then add it to the beginning of the linked list.

Using the ‘private getNode’ helper, this allows us to traverse the bucket chain to look for a matching ID (a duplicate check). This ensures that there’s only one record of a film per ID. Removing film follows the same pattern (find the node by tracking the predecessor and relink the predecessor to attach to the node after the removed node).
Instead of storing production companies and countries in a plain array, they were stored as MyDynamicArray fields on each MovieNode. Each entry is added incrementally, and the total count per film isn’t known when the node is first constructed. A dynamic array is preferred over a fixed-size array, as it doesn’t waste space or require reallocation; it grows as more entries are added, so memory is proportional to actual data.
Film collections required for a new design decision: a collection group of multiple films that can be encountered in some order during loading, so collections can’t be embedded directly in a single film node. So a separate MyDynamicArray of CollectionNode objects was maintained in the Movies class.
Every CollectionNode contains a dynamic list of film IDs associated with the collection’s metadata. A linear search is done over the array when a lookup is done by collectionID. But as the number of unique collections is expected to be small, this search is reasonable and avoids the complexity of the second hash table.

Credits Store:

The Credits store links every TMDB film ID to cast and crew arrays for that film. Its structure resembles the ‘Movies store’ (a hash table with the same capacity with separate chaining); each CreditNode holds a film’s whole cast and crew data. Again, it has the same performance as Movies, lookup by filmID is O(1) on average. A design choice specific to Credits is that some methods require Person-based queries, e.g., a cast member’s films or returning all unique cast members. There was no secondary index using person ID added, as it requires another parallel hash table (using personID) maintenance (and needs synchronisation with every addition and removal).
Instead we focus on searching all the credit nodes; although O(n*k) is the worst case (where n is number of films and k is average cast/crew size), there’s a finite cast per film (and there’s approx. 1000 films). So the actual search time seems small enough to be practical; the simplicity and memory efficiency had higher weighting over optimal worst-case time for this decision.
Also, sorting is needed for two getters: ‘getFilmCast’ (ordered by billing order) and ‘getFilmCrew (ordered by person ID). Selection sort was used on a defensive copy of a stored array: it had simplicity, and the cast and crew array is small per film (i.e., tens of members at most), so the O(n2) cost becomes negligible in practice. A defensive copy is sorted to prevent the original data from being mutated by the query.

Ratings Store:

A more complex design problem for the Ratings store was present. (userID, movieID) pair is used to identify a rating, so the solution needs to search ratings efficiently in two ways: all ratings by a given film and all ratings by a given user. A single hash table would only allow for one search to be performed efficiently, and a full scan for another direction. So the solution was using two parallel hash tables: a userHashTable using userID and a movieHashTable using movieID.

Every RatingNode has two next pointers (nextUser and nextMovie), allowing the same node object to be in two linked-list chains at once (one for each table). This prevents data duplication, as both tables reference the same physical node, so updating the rating's value (using the set method) takes place in both ways, with no extra effort. Just like in the movie store, insertion uses the same approach, but two operations are needed (one for each hash table). Two unlinking operations are needed for a deletion; refactoring allows for the 'removeFromMovieHashTable' helper method to handle removal from the movie table and keep the 'remove' method clean.
Another helper 'getRatingNode' searches only the user table, but correctly identifies a unique rating by matching the userID and movieID (despite many users (and their ratings) sharing the same hash bucket).
Methods like 'getMostRatedMovies' and 'getTopAverageRatedMovies' need aggregation across all ratings. Using parallel MyDynamicArray instances, it allowed per-film counts and sums to accumulate during a single search, then again used selection sort to sort the results. Calculating averages before storing prevents repeated division during comparisons.

Supporting Data Structure: MyDynamicArray:

As the coursework Java spec penalised the use of MyArrayList, MyDynamicArray was implemented as a custom resizable array, satisfying the IList interface. It was an improved version of MyArrayList for the tasks ahead. MyDynamicArray uses an initial capacity of 10 (instead of 100), which is more appropriate for small per-node lists (companies, countries, film IDs in a collection) and is created throughout the stores. MyDynamicArray uses a growth factor of 1.5 (rounded to the nearest integer) (instead of 2), which reduces peak memory overhead when the structure grows larger.
Another extra feature is when 'size' drops below a quarter of capacity following the remove call, then the array capacity is halved (reclaiming memory); this prevents leftover large empty arrays (when entries are deleted from stores).
To prevent boxing and unboxing overhead occurring (when each element is retrieved via get and cast), two conversion helpers (toIntArray and toFloatArray) were added to produce primitive arrays from MyDynamicArray. These methods were frequently used in stores to convert internal dynamic arrays to required int[] or float[] primitive arrays.

Conclusion:

In conclusion, separate chaining hash tables with a capacity of 1999 (prime) was the core for efficient access for all 3 stores. A single hash table using film ID as hashcode used for Movies and Credits stores; two hash tables to allow for two-way queries were used for Ratings stores, with no data duplication.
A dynamic array class (different from MyArray) by having a smaller initial capacity, a moderate growth factor, automatic shrinking, and supports many different lengths per-node lists across the stores. Overall, an O(1) average-case performance is obtained for the most frequent operations from the design choice, and keeps memory proportional to the actual data loaded.